Temporal Referent Integrity: A Controlled Diagnostic for Referential Re-resolution in Tool-Using Agents

Anonymous submission

Abstract

Refreshing external state does not by itself license a tool-using agent to change the entity denoted by an earlier reference. Post-binding target drift has been documented; our contribution is an instruction-timing-controlled evaluation variable that distinguishes legitimate deferred resolution from discourse-inconsistent substitution. We formalize this distinction as *temporal referent integrity* (TRI). Identifying it requires an eligible opportunity, an observable initial binding, and a Preserve/Reevaluate contrast; observing the executed mutation is additionally required to establish a wrong-entity consequence. On two frozen controlled diagnostics, Always-Lock and Always-Reevaluate both score 100% on Stable controls but zero changed-winner PairAcc, whereas Generic controllers selectively substitute the refreshed winner after correct binding. In replayed Qwen/GLM v7 outputs, all 43 and 38 eligible substitutions, respectively, become wrong-entity SQLite writes. Compile-then-act (CTA) reduces this specific drift but does not solve all target errors and is not the unique implementation. Public audits find zero strict native opportunities in the pinned versions of three suites under our strict checklist, and the controlled failure does not reproduce in our lower-intervention external agents. TRI is therefore a controlled, model- and controller-conditional diagnosis of an evaluation gap in these regimes, not a natural prevalence estimate or a universal controller claim. A workflow-grounded task grammar makes the required selector, refresh, and mutation opportunity explicit.

Introduction

Consider two requests issued before the same mailbox refresh:

> (1) "Choose the highest-priority unread email now. Refresh the mailbox, then reply to it."
>
> (2) "Refresh the mailbox first. Then choose the highest-priority unread email and reply."

Suppose email A is highest priority before refresh and email B afterward. The environment transition is identical, but the correct target differs. Request (1) establishes A as an identity commitment; request (2) deliberately leaves a description to be evaluated in the new state. Acting on B in (1) is not stale-state correction. It is a wrong-entity action.

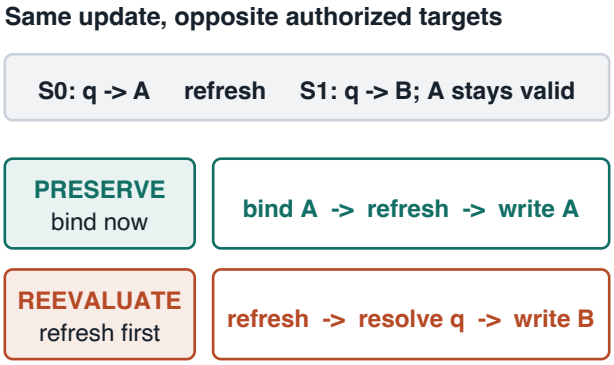

Figure 1: Matched TRI minimal pair. Both rows share q, S_0, S_1, and the mutation, but discourse changes resolution time. $U(q)$ is unresolved and $B(e)$ is bound: Preserve must write A, whereas Reevaluate writes refreshed winner B.

This distinction is easy to lose in LLM agent controllers. A controller may preserve the entire transcript, append the latest observation, and ask a downstream model to act. All task-relevant observations may remain in context, yet the model can still apply the old selector to the new state. Conversely, always retaining an old ID breaks genuinely dynamic references. Reliable tool use therefore needs *selective invariance*: world facts should update while some target denotations remain fixed, others are reevaluated, and invalid commitments follow an explicit fallback policy.

We call this property *temporal referent integrity* (TRI). Here "authorization" means instruction-conditioned resolution timing—whether discourse resolves the target before or after the refresh—not access control or user permissions. TRI is not ordinary entity memory. Remembering an ID does not say whether that ID is a committed referent or merely the previous output of a dynamic selector. Nor does existence imply action validity: an email may remain present but cease to be replyable. Finally, the selector that established identity is not necessarily a persistent validity predicate. A branch chosen because it was default can stop being default while remaining the correct branch on which to run checks.

The central evaluation question is not whether an agent can retain an ID or select a new winner, but whether the benchmark can distinguish the two policies when discourse changes the intended resolution time. We treat TRI primarily as an evaluation-identifiability problem rather than a new general-purpose agent architecture.

Our contributions are threefold:

- We isolate instruction-conditioned re-resolution timing as a benchmark variable distinct from entity binding, memory, and action validity, and state the observability conditions needed to identify discourse-inconsistent substitution and its executed consequence.
- We provide a workflow-grounded Preserve/Reevaluate diagnostic whose matched members share states, selector, action, and transition, then audit how aggregate, Stable-only, and one-sided scoring can reward opposite unconditional policies.
- Across frozen diagnostics, Generic controllers substitute the refreshed winner after correct observable initial binding, and deterministic replay turns those substitutions into wrong-entity writes. Non-unique controller probes and external audits delimit the finding: it has no unique implementation-level remedy and does not establish natural prevalence.

Novelty boundary. Prior work established that a bound target can drift. Our contribution is the orthogonal instruction-timing contrast and the evaluation conditions needed to distinguish legitimate deferred resolution from discourse-inconsistent substitution; we do not claim the first observation of drift or a new general runtime architecture.

Related Work

Reference, binding, and memory. Discourse theories distinguish referents from changing descriptions and salience (Kamp and Reyle 1993; Grosz, Joshi, and Weinstein 1995); coreference work recovers textual identity links (Lee et al. 2017; Joshi et al. 2020; Dasigi et al. 2019). Entity Binding Failures studies wrong initial tool arguments (Babu and Indukuri 2026b); TRI instead conditions on correct $bind(r, S_0)$ and asks whether S_1 licenses a transition. Agent memory and entity tracking retain facts or traces (Tang et al. 2026; Zhang et al. 2024; Packer et al. 2023; Sumers et al. 2024; Yao et al. 2023); correctly tracking the new selector winner still does not authorize acting on it. Temporal Blindness asks when time should trigger a new observation (Cheng et al. 2026); TRI asks what that observation may revise. Prospective-memory evaluation instead asks whether a delayed intention fires at a future cue (Liu and Gabriel 2026). Evolving-environment benchmarks study belief and memory adaptation (Ji et al. 2026; Xu et al. 2026); TRI studies the propagation boundary from belief updates to an already resolved action referent.

Binding Drift. The closest neighbor already studies a correct initial binding being silently replaced later in a workflow, and evaluates locking and re-verification (Babu and Indukuri 2026a); post-binding drift itself is therefore not our discovery. TRI's increment is a complementary evaluation variable: whether the user authorized resolution before the update or deferred it until afterward. Its symmetric Preserve/Reevaluate pairs hold the state transition and action fixed while requiring opposite targets, so a preserve-only evaluation cannot identify this variable. A persistence-only benchmark detects replacement but cannot label it discourse-consistent because it omits opposite-gold re-resolution cases; TRI adds that test. We reproduce Binding Drift's deterministic offline harness, but its oracle re-verifier reads a gold target and its Entity Lock corresponds to our Always-Lock extreme. Its LLM-based self/cross re-verifiers resolve the original step-1 referent and do not expose a separate user-specified deferred query. Initial-binding correctness and transition authorization are thus orthogonal axes; TRI conditions on the former to isolate the latter. Our author adaptation supplies the complete temporal instruction rather than Binding Drift's short referent interface; it is not an official LLM-sweep reproduction. Because it sees S_1 but neither S_0 nor the resolved old ID, we treat it as an interface audit rather than an information-matched performance baseline.

Axis	Binding Drift	TRI
Gold semantics	Primary carry target remains fixed	Preserve/Reevaluate require opposite targets
Initial ID	Step-1 binding is scored	Initial ID must be observable and correct
Re-verifier input	Original referent plus later state	Matched history or compiled timing decision
Verdict	Persistence/repair	PairAcc plus conditional substitution

Table 1: Closest-neighbor boundary. TRI adds an instruction-timing-controlled evaluation axis; it does not claim the first observation of post-binding replacement.

Tool agents and executable constraints. Tool-use benchmarks cover APIs, web environments, and stateful interaction (Schick et al. 2023; Li et al. 2023; Qin et al. 2024; Patil et al. 2023; Liu et al. 2024; Zhou et al. 2024; Lu et al. 2025; Trivedi et al. 2024; Yao et al. 2024; Barres et al. 2025; Sierra Research 2026; Ruan et al. 2024). The three suites audited here do not expose gold labels for strict post-binding Preserve/Reevaluate opportunities; broader tool-use benchmarks commonly emphasize task completion instead of this contrast. LedgerAgent and Bounded Autonomy add typed state or action contracts (Uddin et al. 2026; Sohail and Haider 2026); action validity still cannot determine which valid entity discourse licensed. AgentSpec and policy autoformalization compile or enforce rules, VIGIL monitors temporal and value-flow constraints, and Cordon stages multi-step effects before commit (Wang, Poskitt, and Sun 2025; Mondl, Maisel, and Brock 2026; Li et al. 2026; Chen et al. 2026). They can enforce known rules but neither infer resolution timing nor supply matched gold. Structured generation can enforce syntax (Beurer-Kellner, Fischer, and Vechev 2023; Willard and Louf 2023; Khattab et al. 2024), while our clustered min-

imal pairs follow diagnostic and contrast-set methodology (Ribeiro et al. 2020; Gardner et al. 2020). Complementary semantic conditions have similarly exposed one-sided LLM heuristics and correction trade-offs in aspectual NLI (Ma and Miyao 2026); TRI applies the diagnostic logic to post-binding control and executed actions, not lexical aspect or static entailment.

Temporal Referent Integrity and Evaluation Identifiability

Let H_t be interaction history, S_t world state, r an action-target reference, and q_r its selector. Referential control state is either an unresolved query $C_t(r) = U(q_r)$ or a bound commitment $C_t(r) = B(e)$. Language and history induce a referential transition decision $\Gamma_{t+1}(r, e \to e')$; a world update alone does not:

$$S_t \neq S_{t+1} \not\Rightarrow C_t(r) \neq C_{t+1}(r). \tag{1}$$

TRI deems a distinct transition $B(e) \to B(e')$ valid only if $\Gamma_{t+1}(r, e \to e')$ holds. Thus a selector can be evaluated to establish $B(q_r(S_0))$, preserved across refresh, or left as $U(q_r)$ for evaluation in S_1. Returning from $B(e)$ to query evaluation requires the discourse support encoded by Γ; a refreshed selector winner is world evidence, not that support.

We audit workflows through an OBSERVE–BIND–REFRESH–WRITE grammar: the target is resolved from an observed collection, an intervening state transition occurs, and a later entity-directed mutation consumes the target. Public-benchmark coverage is reported by which of these slots, and which authorization contrast, is actually observable.

Requirements for an identifying evaluation. Opportunity coverage requires a binding, an intervening update, a changed selector winner, a still-valid old target, and a later entity-directed action. Binding observability requires the trace or evaluator to expose the initial ID so initial-binding errors are not confused with post-binding substitution. Authorization discriminability requires matched Preserve and Reevaluate cases under the same transition, so Always-Lock and Always-Reevaluate cannot both look successful. These first three conditions identify discourse-inconsistent substitution. Mutation or consequence observability is an additional requirement when the claim concerns an executed wrong-entity write: the evaluator must expose a state diff or equivalent target-level consequence. These are evaluation conditions, not claims that every benchmark or deployed workflow satisfies them.

State–authorization separation. Updating S_t alone does not mutate a bound $C_t(r)$. History may contain enough discourse evidence to infer the permitted transition, but information availability does not imply that a model-mediated controller will operationalize or enforce it reliably. TRI concerns the control effect of an update, not whether the new world state is known correctly. Explicit authorization state is one implementation; the formal requirement is access to a discourse-sensitive transition decision, not a particular field or serialization.

For execution, $V_a(e, S)$ denotes action-relative validity and π an invalid-target policy. Our lifecycle record $L = (m, e, q, V_a, \pi)$ is one implementation, not the definition of TRI. Under PRESERVE, a valid e remains the target and an invalid e follows π; under REEVALUATE, the target is $q(S_1)$. Action validity never by itself authorizes substituting a different valid entity. TRI-v3 is the original paired diagnostic and v7 its larger new-schema, new-state replication; the conditional-policy v4 extension is supplementary.

Identifiability observation. Consider a Preserve/Reevaluate minimal pair with identical S_0, S_1, q, and a, where $e = q(S_0)$, $e' = q(S_1)$, and $e \neq e'$. The authorized targets are e and e', respectively. Any deterministic post-refresh policy of the form $f(e, q, S_1, a)$, containing neither the original history nor a compiled transition authorization, is wrong on at least one member. Both provide f the same tuple, hence the same output, which cannot equal both distinct authorized targets. Always-Lock and Always-Reevaluate instantiate the two complementary extremes measured in our policy controls. This elementary observation motivates the diagnostic design and the need for a discourse-sensitive transition decision; it is not claimed as a theoretical lower bound or as evidence for our particular serialization.

Outcomes. We separately report exact target/final state, wrong-entity writes, invalid attempts, and unnecessary rejection; always rejecting avoids writes but fails every actionable task.

Policy Controls and Controller Probes

We use three probes to test whether an evaluation rewards the correct discourse-conditioned transition. Historical CTA is a minimal scalar realization: it emits binding time, selector, and a bound ID before refresh, without a deterministic gate. Lifecycle is a typed, auditable realization that adds a validity/fallback contract. Lifecycle-Gated is an execution probe that enforces this contract at mutation time. The typed record emits

```
reference_mode; bound_target_id;
selector; invalidity_policy.
```

Both compilers use discourse order: "identify, refresh, act on it" preserves identity even without words such as "same," while "refresh before deciding" reevaluates. It resolves a concrete bound ID only for preserved references.

The executor then refreshes the environment. If the compiled mode is PRESERVE, a deterministic gate checks the bound entity against action preconditions. The gate emits the bound ID when valid, returns $\perp$ under a reject policy, or invokes selector resolution only when the compiled fallback authorizes it. If the mode is REEVALUATE, the LLM actor selects from S_1. Thus a preserved commitment is a controller constraint, not advice that a second model may reinterpret. The gated Preserve branch needs one model call rather than two.

The complete controller procedure and model-facing interfaces are provided in the supplement. Gold targets and benchmark selector fields are never provided to the compiler or actor.

Information fairness. Compilers see the instruction, S_0, action schema, and planned refresh, never gold targets or generator selector fields. Generic retains the instruction, selected ID, snapshot, selector, action, and preconditions for its second-call actor. CTA, Lifecycle-free, and Generic each use two calls; Lifecycle-Gated skips the actor for a valid Preserve branch. Full prompts, interfaces, and executor-level oracle checks are supplementary.

Experimental Setup

Controlled task construction. Each domain specification defines an entity table, a natural-language selector, its ranking or Boolean criterion, an action, and explicit action preconditions. The generator first evaluates the selector on S_0 to obtain e_0, applies a controlled transition, and evaluates it on S_1 to obtain e_1. A language template determines whether the action target is e_0, e_1, $\perp$, or a conditionally guarded choice. Paired tasks share the domain, both states, selector, action, and schema; only discourse order and anaphoric language change. This prevents domain difficulty or target frequency from explaining the preserve/reevaluate contrast.

Workflow-grounded synthesis. The generator is organized as a four-event workflow grammar rather than isolated selector questions: OBSERVE a user-visible collection, BIND one stable ID, apply an exogenous or synchronization-style REFRESH, and execute a target-specific WRITE. Domain templates fill these slots with ordinary operations (reply to a message, comment on a reminder, update a project or expense, change an inventory record, or deploy a selected artifact). The same-role replacement requirement is added only when the application exposes both a stable ID and a ranking or eligibility selector. This yields a crossed factorial 2×2 core (Preserve/Reevaluate $\times$ Stable/Flip), while Remove, Invalidate, and name collision are orthogonal stressors. We report task-structure coverage in the controlled inventory and audited public suites as design diagnostics, not as estimates of how often TRI occurs in deployment.

The referential core is a crossed diagnostic: resolution mode is PRESERVE or REEVALUATE, while the refreshed selector winner is unchanged (Stable) or changed (Flip/name collision). Stable controls test ordinary execution without a target conflict; changed-winner pairs require opposite targets under identical states. Remove and Invalidate form a separate execution-policy slice because anchored items may require rejection rather than an ID.

The five transition types serve different diagnostic roles. *Flip* changes which valid entity wins the selector. *Stable* leaves the winner unchanged and detects controllers that overreact to any refresh. *Remove* deletes the pre-refresh entity. *Invalidate* keeps its ID present while changing an action-precondition field, separating existence from validity. *Name collision* gives another entity the same display label while changing the selector winner, testing whether controllers preserve stable identity rather than surface aliases. Across anchored and dynamic language, these updates create matched cases where naive "always old" and "always new" policies have complementary failures.

Blind human construct validation. One adult volunteer naturally rewrote 50 sampled scalar instructions. Three other adults, uninvolved in TRI design, independently labeled 100 randomized original/rewrite items using opaque IDs without condition metadata or gold answers. All four provided informed consent. Two received modest one-time honoraria; two were unpaid, and payment was not response-contingent. No formal institutional review or exemption determination was obtained. Annotators selected an entity, REJECT, or CLARIFY; confidence was optional. We report majority–gold alignment, agreement statistics, and a consensus-only sensitivity analysis. The study covers only the single-target scalar core and excludes guarded, multi-target, and nested-reference extensions.

Stage and write-consequence reports use a frozen mechanism-oriented error taxonomy, detailed in the supplement, that separates temporal rebinding, premature binding, invalid processing, unnecessary rejection, and selector-grounding errors.

Symmetric evaluation and component audit. Figure 2(a) visualizes the frozen v3 changed-winner core. The primary estimand is the pre-specified Generic-versus-Lifecycle-Gated comparison; validity gates, untyped plans, Lifecycle-free execution, exact historical CTA, and the handcrafted rule are secondary audits. The v3 method comparison was the original primary and was frozen before calls; the identifiability analysis is a post-primary zero-API reanalysis; v7 is a post-primary replication frozen before its calls; and Rule v2 is post-hoc. Their full component table and protocol chronology are in Supplementary Material.

TRI-v3 language clusters. The primary test contains 160 tasks in 20 frozen language-template clusters, balanced across reference styles and transition types. Data, runner, hashes, inference settings, and the cluster-bootstrap estimand were frozen before model calls; the complete matrix and chronology are provided in Supplementary Material.

Schema transfer under shared language templates and guarded policies. The 80-task transfer set crosses the same templates with four unseen schemas; a separate 40-task TRI-v4 extension tests action-validity versus selector-match guards. Both are secondary extensions, with inventories and guard definitions in Supplementary Material.

Direct-neighbor baseline. Generic Structured Ledger stores the initial ID, entity snapshot, selector, action, and preconditions but omits reference mode and fallback semantics; its actor still receives the original instruction and refreshed state. Historical CTA is rerun unchanged as a neighboring pre-refresh commitment baseline. The full component comparison, including the untyped planner, appears in Supplementary Material and prevents us from claiming that the full typed tuple is necessary for scalar tasks.

For a direct policy contrast, we also evaluate two deterministic extremes without model calls. Always-Lock + validity retains the S_0 ID and rejects it only when the refreshed action preconditions fail; Always-Reevaluate discards the binding and applies q to S_1. These are conceptual controls rather than implementations of the Binding Drift base-

lines. On the frozen 160-task inventory both reach 96/160 (60.0%), but their mode slices are complementary: Always-Lock reaches 80/80 anchored and 16/80 dynamic, whereas Always-Reevaluate reaches 16/80 anchored and 80/80 dynamic. The contrast is a diagnostic of selective re-resolution, not evidence that either unconditional policy is a competitive agent controller.

Model-facing SQLite trajectories. The 40-task SQLite subset covers all templates, domains, modes, and update types. After refresh, the gate or actor supplies a target to an action-validating mutation tool; we record final state, wrong writes, invalid attempts, and collateral modifications. This is a controller-orchestrated execution consequence test, not an autonomous external benchmark; the complete trace protocol is in Supplementary Material.

Models, inference, and auditing. Qwen3.5-122B-A10B and GLM-5.1 use the same temperature-zero OpenAI-compatible protocol; a post-primary DeepSeek-V4-Pro run provides a third-family replication. Smoke gates, retry policy, attempt accounting, and parse/API failure handling are recorded in Supplementary Material.

Statistical estimands. The primary estimand is Lifecycle-Gated minus Generic accuracy on the Qwen v3 language clusters, with cluster bootstrap rather than row-level independence. We report task accuracy, matched-pair accuracy, and conditional substitution separately; the latter requires a correct observable initial ID, a completed refresh, a surviving/action-valid old target, and a distinct refreshed winner. Metric definitions, intervals, and secondary tests are in Supplementary Material.

Reproducible run discipline. Frozen data, protocols, runners, hashes, raw outputs, retries, stage errors, SQLite traces, and source-derived reports are included in the anonymous artifact. Post-primary and post-hoc analyses are labeled explicitly; private returns and credentials are excluded.

Results

Diagnostic Results

Identifiability. Figure 1 defines the task contrast, while Figure 2 exposes false favorable verdicts. Always-Lock and Always-Reevaluate tie at 60.0% on v3 and 66.7% on v7, both reach 100% on Stable, and both score zero changed PairAcc. On v7, Reevaluate-only evaluation rates GLM and DeepSeek Generic at 95.0% and 100.0%, although changed PairAcc is only 15/80 and 17/80 and conditional substitution is 38/80 and 59/79. Qwen shows the same direction at lower accuracy (7/80 PairAcc; 43/72 substitutions). Aggregate, Stable-only, and one-sided scores therefore cannot identify selective re-resolution. This is a post-primary, zero-API reanalysis of frozen outputs. CTA obtains 31/80, 66/80, and 64/80 changed PairAcc with zero eligible substitutions for Qwen/GLM/DeepSeek; its remaining errors are binding, grounding, or execution errors. Table 2 summarizes the frozen v7 outcomes.

Model / ctl.	PairAcc	Cond. subst. ↓	Core/all writes
Qwen / Gen.	7/80	43/72	43/44
Qwen / CTA	31/80	0/71	0/8
GLM / Gen.	15/80	38/80	38/38
GLM / CTA	66/80	0/70	0/14
DeepSeek / Gen.	17/80	59/79	59/60
DeepSeek / CTA	64/80	0/70	0/17

Table 2: Frozen v7 diagnostic outcomes. PairAcc uses 80 matched changed-winner pairs per row. Conditional substitution requires correct observable initial binding, completed refresh, a surviving action-valid old target, and a distinct refreshed winner. The last column reports strict-core substitution writes / all wrong writes in the 240-task deterministic SQLite replay, so CTA's non-core errors remain visible.

Actionable referential core and reject policy. The pre-specified v3 E2E total mixes 128 tasks with an actionable entity target and 32 anchored remove/invalidate tasks whose gold is the author-specified REJECT outcome. On the actionable core, Generic reaches 95/128 (74.2%) for Qwen and 93/128 (72.7%) for GLM; Historical CTA reaches 126/128 (98.4%) and 127/128 (99.2%), while Lifecycle-Gated reaches 125/128 (97.7%) and 128/128 (100.0%). The separate reject-policy slice is 8/32 and 22/32 for Generic, 26/32 and 27/32 for CTA, and 32/32 for Gated. Because human agreement is weaker on rejection, these policy scores are not treated as equally supported referential judgments; full intervals are supplementary.

Conditional substitution and write consequences. Generic fails most when a valid bound entity no longer wins the selector. Conditioning on its stored `selected_entity_id` being correct, Qwen drifts on 15/16 anchored flips and 14/16 name collisions; GLM drifts on 3/16 and 7/16, while Stable controls show none. This directionality rules out random target error as a complete explanation.

The separately frozen v7 replication contains 240 core tasks over ten unseen schemas and 40 state instances; every old target remains present and action-valid. After a correct initial binding, Generic selects exactly the distinct refreshed winner on 43/72 Qwen and 38/80 GLM changed-winner opportunities, versus zero for matched CTA/Gated. A later DeepSeek run finds 59/79 versus 0/70 for Generic/CTA. On the same tasks where both controllers bind correctly, Generic versus CTA substitutions are 41/66 versus 0/66, 30/70 versus 0/70, and 50/69 versus 0/69. This post-primary shared-eligible audit excludes controller-specific denominator selection; zero observed CTA substitutions do not establish zero population risk. A deterministic in-memory SQLite replay of all frozen v7 model outputs turns every one of these 43, 38, and 59 substitutions into a wrong-entity write; this replay is distinct from the separate 40-task model-facing SQLite experiment in the supplement. The conditional denominator excludes initial-ID, missing-ID, action-invalid, API, and parse failures; Stable errors and all non-core writes remain separate. Generic initial binding

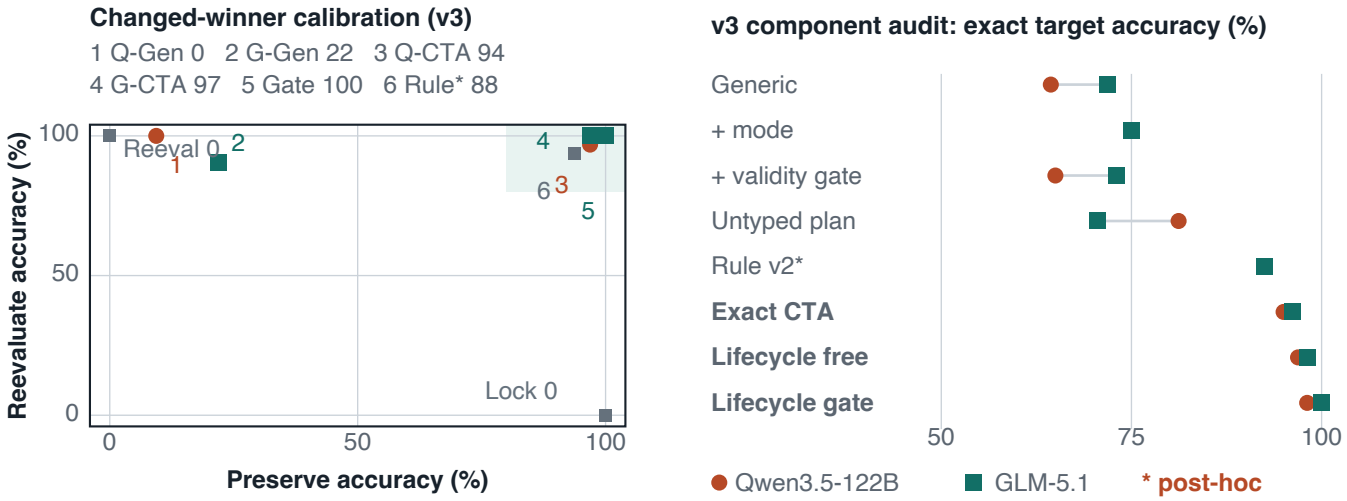

Figure 2: Frozen v3 diagnostic evidence. (a) Preserve versus Reevaluate accuracy on 32 matched changed-winner pairs; the key and corner labels report PairAcc, and unconditional controls occupy opposite corners. (b) The historical component audit is consistent with explicit transition control explaining most of the observed contrast; validity gating adds little. Rule v2 is post-hoc.

is correct on 107/120 Qwen, 120/120 GLM, and 119/120 DeepSeek anchored rows; conditional-substitution intervals are [46.5,72.4], [35.0,61.3], and [62.5,86.1] percent.

The frozen write trace makes the consequence concrete: in a Qwen name-collision task, Generic records `REM-1A` before refresh but writes to refreshed winner `REM-1B`; the SQLite trace labels this `wrong_entity_write`, while CTA writes the discourse-resolved old ID.

Historical primary comparison. The original v3 primary estimand remains the Lifecycle-Gated–Generic difference on Qwen's 20 language clusters: E2E accuracy is 98.1% versus 64.4%, a +33.8-point cluster-bootstrap difference [+18.1,+50.0]. The GLM replication is 100.0% versus 71.9%, +28.1 points [+18.1,+38.1]. Historical CTA reaches 95.0% and 96.2%, respectively. These aggregate totals mix the actionable core and author-specified reject policy and therefore follow, rather than replace, the identifying PairAcc, conditional-substitution, and write-consequence results. Complete E2E, mode-slice, schema-transfer, and model-facing SQLite tables are supplementary.

Leave-domain/template-out analyses and temperature-zero repeats retain the CTA–Generic direction, but incomplete task-level unanimity keeps the claim aggregate and controller-conditional. We distinguish E2E, initial binding, conditional substitution, and executed wrong-write denominators throughout.

Interventions and robustness

What prevents the failure? The matched component audit separates candidate explanations. Mode-only reaches 75.0/75.0%; a validity gate changes Generic by only +0.6/+1.2 points; an untyped pre-refresh plan reaches 81.2/70.6%. CTA reaches 95.0/96.2%, near Lifecycle-free at 96.9/98.1%; deterministic gating adds only +1.2/+1.9 points. An early ID alone is Always-Lock and fails changed-winner Reevaluate. Thus the evidence supports operationalizing an explicit, discourse-sensitive transition decision, not the unique necessity of CTA, typed serialization, pre-refresh timing, or gating alone. Typed state adds auditability and fallback.

Secondary transfer and execution checks are consistent with this interpretation: Lifecycle-Gated improves Qwen schema transfer from 46.2% to 82.5%, while Generic still produces 13/8 wrong writes for Qwen/GLM in the 40-task SQLite run and gating reaches 40/40 final states. These rows remain secondary because selector grounding and invalid-target handling are not TRI itself.

A handcrafted policy narrows the algorithmic claim. After the learned-method analyses, we froze a naive no-LLM event-order rule; unresolved language limits it to 60.6% on v3. After inspecting failures, post-hoc benchmark-aware v2 adds observed event vocabulary and reaches 92.5% on v3, 96.0% on rewrites, and 91.7% on v7. Its v3 intervals overlap CTA. This is not open-language generalization evidence; it shows that an executable discourse-conditioned transition decision, rather than a uniquely learned algorithm, explains success. The information-matched full-history-aware baseline is competitive with CTA for Qwen (69.6 vs. 70.8%) but trails it for GLM and DeepSeek (80.8 vs. 94.2%; 75.8 vs. 91.2%); complete paired results and costs are supplementary. This prevents us from attributing the effect uniquely to pre-refresh compilation.

Human validation and external boundaries

Human validation. Across 100 blinded items, Fleiss' $\kappa = .708$, Krippendorff's $\alpha = .709$, and majority–gold agreement is 86% (91.5% among 94 determinate items). The controller contrast survives human-majority and high-

consensus scoring and 50 volunteer rewrites. Action-invalid REJECT obtains only 55% majority–gold and 25% unanimity, so human evidence supports the Preserve/Reevaluate core more strongly than the normative fallback policy. Against 46 determinate original-item majorities, Generic/CTA reach 69.6/91.3% for Qwen and 73.9/89.1% for GLM; against 48 rewrite majorities, CTA reaches 89.6/93.8%.

External and compositional boundaries. External evaluations do not show a stable controller advantage. In a 24-task ToolSandbox-style pilot, method rankings reverse across Qwen and GLM and gated runs still make wrong writes. A post-hoc strict audit finds GLM Generic rebinding in 3/6 eligible Flip opportunities versus 0/2 Stable controls, but Qwen Generic is 0/6. Four conditions in a frozen 96-task extension yield zero conditional substitution in 64–87 eligible opportunities. The controlled diagnosis has therefore not been reproduced under these lower-intervention external protocols. These are custom, not official or prevalence evidence.

Audits of pinned ToolSandbox (129 families), AppWorld (244 families), and τ^3 (2,449 tasks; 10,832 released trajectories) find zero strict native opportunities under our checklist and a few near-matches. In AppWorld, one action-induced near-match family appears; all 16 auditable subsequent comments retain the same stable ID, while other failures are omissions or upstream/non-gold selection. These are descriptive coverage audits, not claims about other benchmarks or prevalence. In a separate custom two-app study, conditional substitution is 0/24 after correct, timely binding; a lower-intervention addendum without explicit binding language is also 0/28. Its two wrong writes follow synchronization before binding and are classified as tool-order rather than TRI errors.

Composition remains open: a two-refresh Qwen stress test favors Generic over scalar Lifecycle (32/40 vs. 28/40), while a held-out addendum favors role indexing (39/40 vs. 35/40). GLM ITT is 37/40 versus 40/40 scalar because of three transport failures; recovered runs tie at 40/40. We therefore limit the claim to controller-orchestrated, single-refresh, scalar mutations.

Discussion

World evidence and referential control should be separated. Generic carries enough evidence, but leaves unstable whether a selector is executable or merely provenance for a committed ID. It may fail to preserve the dependency from a discourse-resolved binding to the mutation argument, allowing reapplication in a newer state. This controller-level interpretation is neither a unique internal mechanism nor an established sole cause.

The principle is implementation-independent within the diagnostic. CTA, typed lifecycle state, and the post-hoc rule all operationalize the same discourse-sensitive decision. The rule weakens algorithmic novelty but strengthens the control-level diagnosis: the controlled gain does not require method complexity. The present experiment does not establish that CTA generalizes better than rules in open language.

Temporal parsing is a live alternative explanation. The strong post-hoc rule and mode-only gains are compatible with a simpler account: this diagnostic may substantially measure temporal instruction parsing followed by controlled execution. Perfect mode classification by the specialized v3 compiler does not rule out this account; it shows that the authored scalar templates make resolution time recoverable once the controller explicitly extracts it. On unseen schemas, the remaining loss is mainly initial selector grounding rather than mode parsing. The present experiments do not establish that general reference resolution, memory, or entity tracking is the dominant bottleneck beyond this setting.

Gates move, rather than erase, the reliability bottleneck. A gate cannot repair a wrong mode, ID, or guard. It enforces a correct contract, sometimes turning grounding errors into rejection. CTA/Gated produce no core substitution write after correct binding but retain non-core selector errors; enforcement is not a general write-safety guarantee.

Boundary results constrain the design claim. ToolSandbox and multi-referent results show that gates cannot repair bad compilation and scalar contracts do not compose automatically. External nulls reject a universal-failure reading and are compatible with both insufficient observable opportunity and amplification by the controlled interface. The external claim is undercoverage in three audited suite versions, not incidence.

Limitations

Scope and construct validity. TRI is a controlled diagnostic, not a prevalence estimate. It simplifies ambiguity, repair, partial observability, and identity migration. Volunteer rewrites adapt authored tasks rather than independently eliciting natural requests. Human validation covers scalar semantics; low agreement on invalid-target rejection limits fallback claims. The strong rule was tailored after failure inspection and cannot support a natural-language generalization claim. Composition remains open.

Causal attribution and external validity. Temperature-zero repeat runs preserve the paired direction but expose incomplete task-level target agreement, especially for Qwen; they do not characterize all model families. Replications remain synthetic. We reproduced Binding Drift's deterministic assertions, but not its LLM sweep; its oracle reverifier reads gold and Entity Lock matches Always-Lock. The ToolSandbox-compatible positive audit is post-hoc and small, lower-intervention loops are null, and no external study estimates real-traffic prevalence. The nulls leave open both inadequate opportunity coverage and a controlled-interface contribution to the observed effect. Opportunity coverage and controller state affect observability.

Conclusion

World updates need not rebind targets. Matched diagnostics expose whether a refresh licenses re-resolution and its wrong-write consequence. Remedies are non-unique, lower-intervention agents are null, and TRI is an evaluation diagnosis, not prevalence or safety.

References

Babu, R. S.; and Indukuri, S. 2026a. Binding Drift in Multi-Step Tool-Augmented Agents. arXiv:2607.18316.

Babu, R. S.; and Indukuri, S. 2026b. Entity Binding Failures in Tool-Augmented Agents. arXiv:2606.30531.

Barres, V.; Dong, H.; Ray, S.; Si, X.; and Narasimhan, K. 2025. τ^2-Bench: Evaluating Conversational Agents in a Dual-Control Environment. arXiv:2506.07982.

Beurer-Kellner, L.; Fischer, M.; and Vechev, M. 2023. Prompting Is Programming: A Query Language for Large Language Models. In *Proceedings of the 44th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 1946–1965.

Chen, Z.; Liu, H.; Xu, D.; Dong, D.; Li, J.; Pu, B.; and Zhai, J. 2026. Cordon: Semantic Transactions for Tool-Using LLM Agents. arXiv:2606.17573.

Cheng, Y.; Moakhar, A. S.; Fan, C.; Hosseini, P.; Faghih, K.; Sodagar, Z.; Wang, W.; and Feizi, S. 2026. Your LLM Agents Are Temporally Blind: The Misalignment Between Tool Use Decisions and Human Time Perception. Findings of ACL 2026, arXiv:2510.23853.

Dasigi, P.; Liu, N. F.; Marasović, A.; Smith, N. A.; and Gardner, M. 2019. Quoref: A Reading Comprehension Dataset with Questions Requiring Coreferential Reasoning. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 5925–5932.

Gardner, M.; Artzi, Y.; Basmova, V.; Berant, J.; Bogin, B.; Chen, S.; Dasigi, P.; Dua, D.; et al. 2020. Evaluating Models' Local Decision Boundaries via Contrast Sets. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 1307–1323.

Grosz, B. J.; Joshi, A. K.; and Weinstein, S. 1995. Centering: A Framework for Modeling the Local Coherence of Discourse. *Computational Linguistics*, 21(2): 203–225.

Ji, H.; Xiong, K.; Han, S.; Xia, P.; Qiu, S.; Zhou, Y.; Liu, J.; Li, J.; Li, B.; Zheng, Z.; Xie, C.; and Yao, H. 2026. ClawArena: Benchmarking AI Agents in Evolving Information Environments. arXiv:2604.04202.

Joshi, M.; Chen, D.; Liu, Y.; Weld, D. S.; Zettlemoyer, L.; and Levy, O. 2020. SpanBERT: Improving Pre-training by Representing and Predicting Spans. *Transactions of the Association for Computational Linguistics*, 8: 64–77.

Kamp, H.; and Reyle, U. 1993. *From Discourse to Logic: Introduction to Modeltheoretic Semantics of Natural Language, Formal Logic and Discourse Representation Theory*. Kluwer Academic Publishers.

Khattab, O.; Singhvi, A.; Maheshwari, P.; Zhang, Z.; Santhanam, K.; Vardhamanan, S.; Haq, S.; Sharma, A.; et al. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *International Conference on Learning Representations*.

Lee, K.; He, L.; Lewis, M.; and Zettlemoyer, L. 2017. End-to-end Neural Coreference Resolution. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 188–197.

Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 3102–3116.

Li, Y.; Chen, Y.; Wen, H.; Zhang, B.; Liu, H.; Wang, P.; Feng, Y.; and Tian, Y. 2026. VIGIL: Runtime Enforcement of Behavioral Specifications in AI Agent Skills. arXiv:2606.26524.

Liu, G.; and Gabriel, S. 2026. PM-Bench: Evaluating Prospective Memory in LLM Agents. In *Conference on Language Modeling*. ArXiv:2607.12385.

Liu, X.; Yu, H.; Zhang, H.; Xu, Y.; Lei, X.; Lai, H.; Gu, Y.; Ding, H.; Men, K.; et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*.

Lu, J.; Holleis, T.; Zhang, Y.; Aumayer, B.; Nan, F.; Bai, H.; Ma, S.; Ma, S.; Li, M.; Yin, G.; Wang, Z.; and Pang, R. 2025. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 1160–1183.

Ma, B.; and Miyao, Y. 2026. The Imperfective Paradox in Large Language Models. arXiv:2601.09373.

Mondl, A.; Maisel, M.; and Brock, J. H. 2026. Autoformalization of Agent Instructions into Policy-as-Code. arXiv:2606.26649.

Packer, C.; Wooders, S.; Lin, K.; Fang, V.; Patil, S. G.; Stoica, I.; and Gonzalez, J. E. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.

Patil, S. G.; Zhang, T.; Wang, X.; and Gonzalez, J. E. 2023. Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334.

Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; et al. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*.

Ribeiro, M. T.; Wu, T.; Guestrin, C.; and Singh, S. 2020. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 4902–4912.

Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *International Conference on Learning Representations*.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*.

Sierra Research. 2026. τ^3-Bench 1.0.0: Voice, Knowledge, and Task Quality. GitHub software release.

Sohail, S.; and Haider, G. 2026. Bounded Autonomy for Enterprise AI: Typed Action Contracts and Consumer-Side Execution. arXiv:2604.14723.

Sumers, T. R.; Yao, S.; Narasimhan, K.; and Griffiths, T. L. 2024. Cognitive Architectures for Language Agents. *Transactions on Machine Learning Research*.

Tang, Z.; Zhao, Q.; Franco, G.; Wijaya, D.; Mueller, A.; Schuster, S.; and Kim, N. 2026. Do Language Models Track Entities Across State Changes? In *International Conference on Machine Learning*. ArXiv:2605.30233.

Trivedi, H.; Khot, T.; Hartmann, M.; Manku, R.; Li, V. D.; Gupta, S.; Sabharwal, A.; and Balasubramanian, N. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. arXiv:2407.18901.

Uddin, M. N.; Saeidi, A.; Blanco, E.; and Baral, C. 2026. LedgerAgent: Structured State for Policy-Adherent Tool-Calling Agents. arXiv:2606.20529.

Wang, H.; Poskitt, C. M.; and Sun, J. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv:2503.18666.

Willard, B. T.; and Louf, R. 2023. Efficient Guided Generation for Large Language Models. arXiv:2307.09702.

Xu, J.; Li, Q.; Wu, J.; Lan, Y.; Li, S.; Zhou, H.; Jiang, B.; Wang, L.; Wang, J.; Luu, A. T.; Xiong, C.; Park, H. W.; Hooi, B.; and Hu, Z. 2026. EvoArena: Tracking Memory Evolution for Robust LLM Agents in Dynamic Environments. arXiv:2606.13681.

Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2024. Tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.

Zhang, Z.; Bo, X.; Ma, C.; Li, R.; Chen, X.; Dai, Q.; Zhu, J.; Dong, Z.; and Wen, J.-R. 2024. A Survey on the Memory Mechanism of Large Language Model Based Agents. arXiv:2404.13501.

Zhou, S.; Xu, F. F.; Zhu, H.; Zhou, X.; Lo, R.; Sridhar, A.; Cheng, X.; Ou, T.; Bisk, Y.; et al. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*.